ALERT-02: Choosing and Building Detection

> **Series:** ALERT — Alerting Strategy and Design | **Notebook:** 02 of 05 | **Created:** June 2026 | **Last Updated:** 07/30/2026

Overview

Detection is a choice between four mechanisms, and picking the wrong one is the root of most alert noise. This notebook is the **decision framework** — which mechanism for which signal, and why — then it hands off to AIOPS-02 for the build mechanics and SLO-04 for reliability alerting. It deliberately does not re-document the anomaly detector's knobs; it tells you *which* tool to reach for.

Table of Contents

Prerequisites

Requirement	Details
Dynatrace Environment	SaaS Gen3 with the Anomaly Detection app
Prior reading	ALERT-01 (the end-to-end picture)
Build mechanics	AIOPS-02 (anomaly detector setup), SLO-02/04 (SLIs and burn-rate)

1. The Four Mechanisms

Mechanism	What it detects	Owns the detail
OOTB Davis	Latency/error/saturation anomalies, automatically, with seasonal baselines	AIOPS-02 §1
Custom Davis anomaly detector	A business-specific signal Davis does not cover, expressed as a DQL query an analyzer judges	AIOPS-02 §4
OpenPipeline-derived metric	A signal that lives in logs/spans, extracted to a metric at ingest and then alerted on cheaply	OPIPE, AIOPS-02 §6
SLO burn-rate	A user journey burning its error budget too fast	SLO-04

These are not alternatives to rank once — they are a toolkit. Most environments use all four.

2. The Decision

Walk it top-down and stop at the first match:

1. **Is OOTB Davis already detecting it?** Check the problem feed and anomaly detection settings first. If yes → tune sensitivity, do not build anything.
2. **Is it a reliability promise about a user journey?** → an **SLO** with burn-rate alerting (SLO-04). This is the highest-signal option.

3. **Does the signal live in logs or spans, and recur?** → extract an **OpenPipeline metric**, then put a detector on the metric. Cheaper than querying raw data on every evaluation.
4. **Is it a custom condition on existing metrics?** → a **custom Davis anomaly detector** with an auto-adaptive or seasonal analyzer.

The cost — to build and to maintain — rises as you go down. Staying high is the lever on noise (ALERT-01 §2).

3. Anti-Patterns

- **Static thresholds on traffic-correlated metrics.** They alert every off-peak hour and every traffic spike. Use auto-adaptive or seasonal (AIOPS-02 §1). Reserve static thresholds for true hard limits (SLO/contract/capacity).
- **Duplicating Davis.** Before building a custom detector, confirm OOTB Davis or an existing metric event does not already cover the condition. Duplicate detection means duplicate alerts.
- **Querying logs/traces directly in a recurring detector.** Pays query cost on every evaluation, forever. Extract a metric first (FAQ-09, OPIPE).
- **A detector with a bare event template.** No team/zone property means nothing to route on (ALERT-01 §4). Worse, an event template that leaves `dt.smartscape_source.id` unset cannot correlate at all — every firing opens its own problem, and no threshold change fixes it (AIOPS-02 §4, AIOPS-03 §1).
- **Splitting the alert on a high-cardinality dimension.** Grouping a detector by application version, pod name, or HTTP status code turns one condition into one alert per dimension value. Alert volume then scales with your deployment frequency, and alerts strand themselves on entities that no longer exist — a pod replaced an hour ago still carries an open problem. Alert on the aggregate; working out *which* version or pod caused it is a downstream investigation step, not an alerting dimension. (Davis's own multi-dimensional baselining is a different mechanism and does want the dimensions — AIOPS-02 §1.4.)
- **The cloned detector.** In a fast rollout the dominant noise source is not one badly-tuned alert — it is a base detector template copied across teams with its threshold, sensitivity, and routing left unchanged. Two weeks later that team has muted the channel. You cannot review your way out of this one clone at a time; defend at the template, not at each copy.

Make the base template safe by default. The fix is to make the *default* shape adaptive, not static, so the worst case of a copy-paste is a correctly-scoped, loosely-tuned alert — never an unscoped firehose. A base detector template should ship with:

- An **auto-adaptive (or seasonal) analyzer as the default**, not a static threshold — the only inputs a team must supply are the metric/selector and the owning team/area property.
- **No-data does not alert**, so missing telemetry during onboarding stays silent until instrumentation lands.
- A **violating-samples / sliding-window minimum** (e.g. ≥3-of-5) so a single transient spike never pages.
- **Routing bound to the team/area property the template already requires**, so even an untuned clone reaches the right owner.

Forthcoming: a second lever, at a different layer

Rolling out (SaaS 1.344): problem-event trigger delays become configurable — how long a Davis event must persist before it opens a problem. SaaS 1.344 was published 07/27/2026 with a **staged tenant rollout from 07/29/2026**; verify it has reached your tenant before designing around it.

This is **not** a fifth analyzer knob, and it does not replace the sliding-window minimum above. The analyzer parameters decide whether a series is anomalous; the trigger delay decides how long the resulting event must hold before a problem opens. They act on different steps, so they compose — and because the delay applies platform-side, it damps flapping from detectors you do not own, which is exactly the case a template standard cannot reach. Until it arrives, the violating-samples / sliding-window minimum remains the control to rely on for transient-spike noise.

> **Sources:** SaaS 1.344 release notes (DT docs). **Derived:** the "composes rather than substitutes" placement follows from the delay acting on the event-to-problem step while analyzer parameters act on the series.

4. Prototype Before You Commit

Whichever mechanism you choose, develop the underlying query in a notebook first — run it, confirm it returns a sane result over a representative window, *then* wire it into the

detector or SLO. A detector built on a query that silently returns nothing never fires and is worse than no alert, because the team believes it is covered. This notebook-as-scratchpad discipline is covered in AIOPS-02 §4 and SLO-02 §6.

Then deploy relaxed, and tighten later. Once the query is sound, resist configuring the detector at the sensitivity you think you want. Ship it with a high static threshold or a wide deviation margin, watch it for one to two weeks without acting on every trigger, and only then reduce the margin toward the real signal.

The asymmetry is the whole argument: under-alerting for a fortnight is recoverable, whereas a team that muted your channel because the first week was noise is a much harder problem — and they will still be muted when the detector is finally tuned correctly. Alert fatigue is easier to avoid than to undo.

The rigorous version of this makes the observation window explicit rather than informal: deploy the detector with its event type set to `CUSTOM_INFO`, so it is recorded and queryable but never opens a problem, measure how often it actually fires over two to four weeks, and promote it to `CUSTOM_ALERT` only once that number looks sane. AIOPS-02 §4 covers the mechanics and §8 the measuring query.

> **Sources:** Avoid overalerting (DT docs).

5. Hand-Off

Mechanism chosen	Build it in
Tune OOTB Davis	AIOPS-02 §1–§3
Custom anomaly detector	AIOPS-02 §4 (analyzer, tuning knobs, event template)
Metric events / schemas as code	AIOPS-02 §6, AUTOM-05/06
OpenPipeline-derived metric	OPIPE, FAQ-09
SLO burn-rate	SLO-02 (SLI), SLO-04 (alerting)

Then return to ALERT-03 to route what fires.

> **Sources:** Anomaly Detection app (DT docs). **Derived:** the top-down decision order synthesises

> OOTB-first guidance with the query-cost economics in OPIPE/FINOPS.
